TEKNOFEST · Yapay Zekâ Destekli Havayolu Optimizasyonu

ÖZGÜNLÜK, TERİMLER ve JÜRİ SORULARI

Biz ne yaptık · her terim sıfırdan · 45 soru ve cevabı

Takım: Büke

Brief'ten sonuca: hangi adım kimin işi?



Bölüm	İçerik
1 · Biz ne yaptık?	"Model yaptık mı?" — kelimenin iki anlamı · çözücü ne yapar, biz ne yaptık
2 · Özgünlük	Formülasyon · çözüm stratejisi · veri mühendisliği · doğrulama · raporlama
3 · Yapmasaydık ne olurdu?	10 tasarım kararının ölçülmüş alternatifi
4 · Neler bizim değil	Dürüst sınır çizimi
5 · Terimler	MIP · big-M · one-hot · warm-start · MIP-gap · λ cezası · 40+ terim, sıfırdan
6 · Jüri soru bankası	45 soru · problem · model · çözücü · veri · doğrulama · eleştiri

Tüm sayılar projenin gerçek çıktı ve veri dosyalarından hesaplanmıştır.

1 Biz bu projede ne yaptık?

Bu bölüm tek bir soruya cevap veriyor: **"Çözücüyü kullandınız, sizin katkınız ne?"** Jüri bunu soracak. Cevap sandığınızdan güçlü — ama önce bir kavram karışıklığını temizlemek gerekiyor.

1.1 "Model yaptık mı?" — kelimenin iki anlamı

"Model" ne demek?	Bu projede	Açıklama
Eğitilmiş yapay zekâ artefaktı (makine öğrenmesi modeli)	**YOK** — ve olmamalı	Bu bir tahmin problemi değil. Geçmişten öğrenip gelecek tahmin etmiyoruz; kısıtlar altında en iyi kararı arıyoruz
Matematiksel formülasyon (değişkenler, kısıtlar, amaç)	**TAMAMEN BİZİM**	`03_model.py`'nin 1.357 satırı. Brief size düz yazıyla bir iş problemi verdi; onu çözülebilir bir matematiksel yapıya çevirmek işin özüdür

> **Neden bu ayrım önemli?**
> Jüri "yapay zekâ" başlığını görüp bir sinir ağı bekleyebilir. Sinir ağı burada **yanlış araç** olurdu: sinir ağı size *muhtemelen iyi* bir tarife verir, ama hiçbir kısıtı garanti edemez ve "bu en iyisi mi?" sorusunu yanıtlayamaz. MIP ise hem her kısıtı **matematiksel olarak garanti eder** hem de optimale ne kadar yakın olduğunu **kanıtlar**.

1.2 Çözücü ne yapıyor, biz ne yaptık?



Şekil 1.1 — Brief'ten sonuca giden beş adımın dördü bizim tasarımımız.

HiGHS havayolu bilmiyor. Aktarma nedir, hub nedir, uçak rotasyonu nedir — hiçbir fikri yok. Ona ne verirseniz onu çözer. Bu yüzden:

Çözücü, sorduğunuz sorunun cevabını verir. Bütün maharet doğru soruyu sormaktır.

Yanlış formüle ederseniz çözücü size **yanlış sorunun kusursuz optimal cevabını** verir — ve bunu fark etmenizin hiçbir yolu olmaz. Örneğin J'yi 24 saat eksik hesaplasaydınız, çözücü "yenmediğiniz rakipleri yendiniz" der ve bunu mükemmel bir matematikle kanıtlardı.

"Hesabı çözücüye yaptırdık" demek, "*binayı vinç yaptı*" demek gibidir. Vinç ağırlığı kaldırır; neyin nereye konacağına mühendis karar verir.

2 Özgünlük — beş başlıkta

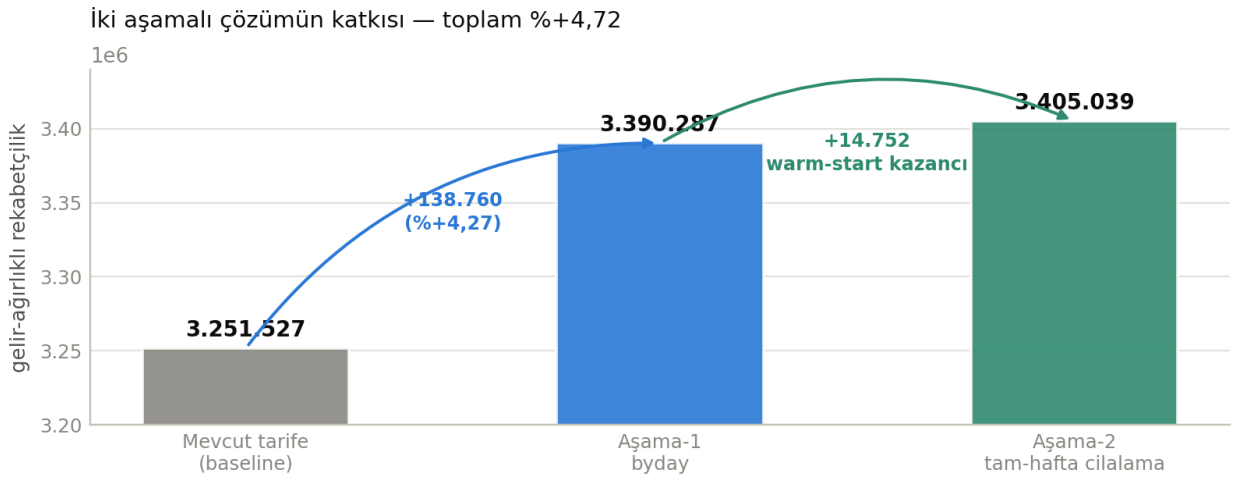
2.1 Formülasyon kararları (asıl entelektüel katkı)

Brief'te "karar değişkenini olay bazında tanımlayın" yazmıyor. Bunların hepsi tasarım tercihidir ve her birinin bir gerekçesi var:

Karar	Neden önemli?	Kodda
Olay soyutlaması — karar birimi bağlantı değil, `(uçuş, rol, saat)`	Bir uçuş 20 bağlantıda geçiyor. Bağlantı başına karar verseydik aynı uçağa 20 farklı saat atanabilirdi. Bu tasarım tutarsızlığı **yapısal olarak imkânsız** kılıyor	`03:232–247`
Tek-sıcak (one-hot) zaman kodlaması	Tarife-değişim cezası `Σ|v−cur|·x` olarak **tam doğrusal** yazılabiliyor — sıfır ek değişken, sıfır big-M. F kapasitesi de aynı değişkenlerden kuruluyor	`03:348–354`
`z` slot yapısı (Kısıt C)	"1. bağlantı çok değerli, 5.'si değersiz" fikrini monoton doldurma kısıtıyla doğrusallaştırma	`03:402–418`
`y` seçici yapısı (Kısıt D)	Ayrık bir lookup tablosunu (doğrusal olmayan fonksiyon) doğrusal amaç terimlerine çevirme	`03:459–475`
Yön-yerel sıkı big-M	Genel bir "büyük sayı" yerine her bağlantının gerçekten ulaşabileceği aralıktan hesaplanan en sıkı M. Gevşek M = çözücü ağaçta kaybolur	`03:386–390`
Üç yollu `w` elemesi	13.015 (bağlantı × rakip) çiftinin **%42'si için hiç değişken yaratılmıyor**	`03:428–441`
`b_od`'nin `s` ile aynı tabandan türetilmesi	Bu yakalayış olmasaydı mevcut tarife pazarların **~%29'unda** hiçbir şey yapmadan sıralama ödülü kazanacak, iyileşmemiz sahte görünecekti	`03:298–314`

2.2 Çözüm stratejisi (algoritmik katkı)

Bu kısım tamamen bizim tasarımımız, çünkü **çözücü tek başına çözemiyordu**:



Şekil 2.1 — İki aşamalı çözümün katkısı. Aşama-2 tek başına +14.752 puan getirdi ve global optimalite kanıtını üretti.

1. **İki aşamalı matheuristik.** Monolitik 7-gün MIP çözülemiyor (~100k ikili, HiGHS tek bir uygun çözüm bile bulamıyor). Güne-göre ayrıştırma → tam-hafta warm-start hattını biz tasarladık.

2. **Gece-aşan rotasyon koruması** (`compute_overnight_guards`, `03:979–1024`) — en zarif parça. Gün ayrıştırması gece-aşan turları göremez. Mevcut payı statik olarak ikiye bölüp iki **tek-taraflı sınıra** çevirdik; böylece iki gün birbirini hiç görmese bile ihlal *matematiksel olarak imkânsız* hâle geldi. Bağlaşık bir kısıtı, bağlaşıklık yaratmadan garantilemek.
3. **G bandının günler arası zincirlenmesi** + çatışmada **iki aşamalı hedefli gevşetme** (önce yalnız rotasyon-korumalı uçuşların bandını bırak, sonra tam gevşet) — `03:1066–1084`.
4. **Warm-start'ta `y`'nin uygunluktan değil byday'in gerçek sunumundan alınması** (`08:77–80`). Bunu fark etmeseydik HiGHS başlangıç noktasını reddedecek ve 08 hiç çalışmayacaktı.
5. **Yeniden yazma bariyeri** (`08:475–485`) — iyi bir çözüm asla kötüsüyle ezilmiyor; iki koşul birden sağlanmadan `output/` dosyalarına dokunulmuyor.

2.3 Veri mühendisliği

Bulgu	Ne yaptık?	Etkisi
24 saat sarması	J'yi hazır sütundan okumak yerine bacak sürelerinden hesapladık + mod-24 çapraz doğrulaması	57.317 satırda 0 tutarsızlık; eski yöntemle 154 satır yanlış olacaktı
Hafta kayması	Aynı operasyon gününün farklı takvim haftalarına yazılmasını hizaladık	130 kopya olay birleşti; aynı uçağa iki farklı saat atanması engellendi
Özdeş kopyalar	Hizalama sonrası birebir aynı kalan satırları attık	157 bağlantının iki kez ödüllendirilmesi engellendi
Amaç evreni eksikliği (F)	`extra_use` — modellenmeyen uçuşları kapasitede sabit doluluk olarak saydık	Model olmayan kapasiteye uçuş yerleştiremiyor
Amaç evreni eksikliği (A2)	`full_role_time` — rotasyon ortağının modellenmeyen tarafını gerçek saatiyle ankorladık	**Gerçek bir A2 ihlali** kapatıldı (doğrulayıcı yakalamıştı)

2.4 Doğrulama — en ayırt edici yanımız

Çift kayıt ilkesi

`05_validate_solution.py` modelin kodundan **tamamen ayrı**. Pyomo import etmiyor, `03_model.py`'ye dokunmuyor, sabitleri (Wdev, τ, Xdev, kapasiteler) kendi dosyasında **yeniden yazıyor**. Girdisi yalnız `derived/` (veri) ve `output/` (çözüm). Baseline'ı bile sıfırdan hesaplıyor.

Böylece modeldeki bir hata, doğrulamada **aynı hatayla örtülemez**.

Ve gerçekten iş gördü. Kod içindeki not (`05:52–60`) şunu kaydediyor:

1. Pencere ±90 dakikaya çıkarıldı, sonuç **%5,34**'e yükseldi — cazip.
2. Doğrulayıcı **gerçek bir A2 ihlali** yakaladı (uçuş 2200/2201, gün 2).
3. Kök neden bulundu: rotasyon ortağının o günkü olayı ρ'suz bir pazara ait olduğu için hiç modellenmiyordu.
4. **Yüksek rakam iptal edildi**, ±60'a dönüldü, kök neden kalıcı olarak düzeltildi.

Sunumda anlatılacak hikâye

"Daha yüksek bir sonucumuz vardı. Kendi doğrulayıcımız gerçek bir kısıt ihlali yakaladı. O sonucu çöpe attık ve hatayı kalıcı olarak düzelttik."

> Bu cümle, bir jürinin bir takım hakkında duyabileceği en güçlü cümlelerden biridir.

Doğrulayıcının bir başka ince ayrıntısı: **fazla-beyan hata, eksik-beyan not** sayılıyor. Yani kendi lehimize olan sapmalar hata kabul ediliyor, aleyhimize olanlar edilmiyor. Ölçüm kendine karşı kurulmuş.

2.5 Dürüst raporlama

- **λ tahminle değil taramayla seçildi** — 5 değerlik Pareto taraması, knee noktası (`03_model.py sweep`, `output/sweep_results.csv`).
- **Duyarlılık analizi, ölçümlerin neyi kanıtlamadığını açıkça yazıyor** — "Wdev=90 ölçümü zaman-limitlidir, fark serbestlik etkisi olmayabilir".
- **Sınırlar sonuçla birlikte yayınlanıyor** — E2'nin kapalı olduğu, G muafiyetlerinin sayısı (1.132), A2 artefaktları: hepsi `results_summary.md`'de.
- **Global MIP-gap %1,18** — optimalite kanıtı; greedy bir yöntemin veremeyeceği tek şey.

3 En güçlü kanıt: "yapmasaydık ne olurdu?"

Katkıyı anlatmanın en ikna edici yolu, her tasarım kararının **alternatifini** göstermektir. Aşağıdaki her satır bir karardır; hiçbiri çözücüden gelmiyor.

Yapmasaydık	Ne olurdu?	Sayısal etki
Kesin J hesabı	Toplam yolculuk süresi hazır sütundan okunurdu; 24 saati aşanlar 24 saat kısa görünürdü → yenmediğimiz rakipleri yenmiş sayardık	**154 satır** yanlış; 29'u bizim, ortalama ρ = 487 olan pazarlarda
`b_od` düzeltmesi	Mevcut tarife ödül tablosunun köşegenine oturmaz, hiçbir şey yapmadan sıralama ödülü kazanırdı → baseline şişer, iyileşme sahte görünürdü	Pazarların **~%29'u** etkilenirdi
Hafta normalizasyonu	Aynı fiziksel uçuş iki ayrı karar olayına bölünür, çözücü ona iki farklı saat atayabilirdi → uygulanamaz tarife	**130** kopya olay
Tekilleştirme	Birebir özdeş satırlar iki ayrı ürün sayılır, aktarma ödülü iki kez yazılırdı	**157** bağlantı
`extra_use`	ρ'suz pazarların uçakları kapasitede sayılmaz, model gerçekte olmayan boşluğa uçuş yerleştirirdi	**79** kova etkileniyor
`full_role_time` ankoru	Rotasyon ortağının modellenmeyen günlerinde A2 kısıtı hiç kurulmazdı	**Gerçek A2 ihlali** (doğrulayıcı yakaladı)
byday ayrıştırması	Tam-hafta modeli tek seferde çözülemiyor — HiGHS incumbent bulamıyor	**Hiç çözüm yok**
Tam-hafta cilalama	Günler-arası koordinasyon kaybı kalırdı ve optimalite kanıtı üretilemezdi	**−14.752** puan + **gap kanıtı yok**
λ cezası	Çözücü amacı değiştirmeyen kaydırmaları da yapardı (dejenerasyon)	Uçuşların **~%90'ı** sıfır kazanç için kaydırılırdı
Bağımsız doğrulayıcı	Wdev=90 denemesindeki gerçek A2 ihlali fark edilmezdi	**%5,34** ihlalli olarak teslim edilirdi

Bu tablo neden bu kadar güçlü?

Çünkü her satır **ölçülmüş**. "İyi iş çıkardık" demiyoruz; "şunu yapmasaydık şu kadar hata olurdu" diyoruz ve sayı veriyoruz. Bir jüri için bu, ezberlenmiş bir sunumdan çok daha ikna edicidir.

4 Dürüst olalım: neler bizim değil?

Bunu söylemek sizi **güçlendirir**, zayıflatmaz. Jüri overclaim'i (abartılı iddia) hemen yakalar; sınırlarını bilen bir takım güven verir.

Bizim DEĞİL	Ne kadarı hazır geldi?	Bizim kattığımız
HiGHS'in algoritması	Dallanma-sınırlama, kesme düzlemleri, sezgiseller — tamamı hazır	Ayarları (determinizm, sezgisel bütçesi) ve **çözülebilir bir model** vermek
Pyomo	Modelleme dili ve çözücü arayüzü hazır	Modelin kendisi — 1.357 satır formülasyon
Warm-start fikri	Literatürde var (*fix-and-optimize* ailesi)	Bu probleme uyarlanması, gece-aşan koruma tasarımı, E1 onarımı, bariyer
Standart OR teknikleri	Azalan getiri, big-M, one-hot kodlama — hepsi bilinen teknikler	**Hangisinin nereye uygulanacağı** ve M'lerin nasıl sıkılaştırılacağı

Jüri "bunu siz mi buldunuz?" derse

"Teknikler standart yöneylem araştırması araçları — big-M, one-hot kodlama, azalan getiri, warm-start. Bizim katkımız bu araçların bu probleme nasıl uygulanacağına dair tasarım kararları: karar biriminin ne olacağı, hangi değişkenin hiç yaratılmayacağı, M'lerin nasıl sıkılaştırılacağı, çözülemeyen modelin nasıl ikiye bölüneceği ve sonucun nasıl bağımsız doğrulanacağı."

5 Terimler — sıfırdan konu anlatımı

Bu bölüm bir sözlük değil, bir **ders**. Her terim üç adımda anlatılıyor: *nedir → neden gerekli → bu projede nerede*. Hiçbir ön bilgi varsayılmıyor.

5.1 Optimizasyonun temel kavramları

Optimizasyon

Nedir? Bir amacı en iyi hâle getirecek kararları bulmak. Günlük hayattan örnek: markete gideceksiniz, 500 lira paranız var, alışveriş listeniz uzun ve bazı ürünler indirimde. "Bütçeyi aşmadan en çok ihtiyacımı karşılayan sepeti kur" demek bir optimizasyon problemidir.

Üç bileşeni vardır ve her optimizasyon probleminde bu üçü aranır:

Bileşen	Markette	Bizim projede
Karar değişkenleri	Hangi üründen kaç tane alacağım?	Her uçuş olayının IST saati
Kısıtlar	Bütçe 500 lira; süt en fazla 2 litre	Uçak rotasyonu, hub kapasitesi, bağlantı penceresi, düzenlilik, yön dengesi
Amaç fonksiyonu	Toplam faydayı en büyük yap	Gelir-ağırlıklı rekabetçilik puanını en büyük yap

Karar değişkeni

Nedir? Değerine *bizim* karar verdiğimiz büyüklük. Veriden gelmez, hesaplanmaz — seçilir.

Neden önemli? Bir problemi modellerken en kritik soru şudur: *neyin değerini seçiyorum?* Yanlış seçerseniz model ya çözülemez ya da anlamsız sonuç verir.

> **Bu projedeki karar değişkeni**
>
> `x[olay, saat]` — "şu uçuş olayı, şu saate atandı mı?" (evet/hayır)
>
> Dikkat: karar değişkenimiz **bağlantı** değil, **olay**. Bu bilinçli bir tercih; nedeni Bölüm 2.1'de anlatıldı: bir uçuş 20 farklı bağlantıda geçiyor, bağlantı başına karar verseydik aynı uçağa 20 farklı saat atanabilirdi.

İkili (binary) değişken

Nedir? Yalnızca **0** veya **1** değeri alabilen karar değişkeni. Bir evet/hayır sorusunun matematiksel karşılığıdır.

Neden 0/1? Çünkü gerçek hayattaki kararların çoğu böyle: *bu uçuş bu saate mi atandı?* (evet/hayır), *bu bağlantıyı sunuyor muyuz?* (evet/hayır), *bu rakibi yendik mi?* (evet/hayır). Bunlara "yarım" cevap veremezsiniz.

Değişken	Sorduğu soru	Kaç tane var?
`x[olay, saat]`	Bu uçuş olayı bu saate mi atandı?	**59.189**
`y[π]`	Bu bağlantı sunuluyor mu?	7.577
`z[pazar, slot]`	Bu pazarın kaçıncı ödül slotu dolu?	7.577
`w[π, rakip]`	Bu bağlantı bu rakibi yeniyor mu?	7.531

Değişken	Sorduğu soru	Kaç tane var?
`β[pazar, rakip]`	Bu rakip yenildi mi?	7.002
`γ[pazar, sayı]`	Bu pazarda tam kaç rakip yenildi?	11.690
TOPLAM		**100.566**

"Tamsayı" neden problemi zorlaştırıyor?

Değişkenler kesirli olabilseydi (0,63 gibi) problem **çok hızlı** çözülürdü — buna doğrusal programlama denir ve saniyeler sürer. Ama "uçuşun %63'ünü 08:30'a ata" anlamsızdır. Tamsayı şartı gerçekçiliği getirir, karşılığında zorluğu da getirir. "Karışık Tamsayılı" ifadesindeki zorluk tam olarak budur.

Kısıt (constraint)

Nedir? Çözümün uyması **zorunlu** olan kural. Kısıt ihlal edilirse çözüm geçersizdir — puanı ne olursa olsun.

Ödülle farkı: Amaç fonksiyonu "bunu yaparsan puan kazanırsın" der; kısıt "bunu yapamazsın" der. Amaç bir **tercih**, kısıt bir **yasaktır**.

Kısıt	Türkçesi	Kaç kısıt
A1	Her uçuş olayına tam bir saat ata	4.553
A2	Dönüş varışı ≥ gidiş kalkışı + uçuş + yer + uçuş	892
B1	Bir bağlantı ancak boşluğu 60–300 dk ise sunulabilir	15.154
C	Ödül slotları baştan doldurulmalı (monotonluk)	7.577
D	Rakip yenme ve sıralama göstergelerinin tutarlılığı	~30.000
E1	Gidiş ve dönüş yönleri dengeli olmalı (±%20)	~8.500
F	Her 10 dk kovada kalkış ≤ 10, varış ≤ 15	1.395
G	Aynı uçuş her gün ±15 dk bandında kalmalı	4.503

Amaç fonksiyonu

Nedir? "İyi"yi tanımlayan formül. Çözücü bu sayıyı en büyük (veya en küçük) yapmaya çalışır.

Amaç = Σ_pazar ρ · (aktarma ödülü + sıralama ödülü) − λ · Σ|kaydırma|

ρ (rho) — pazarın gelir/önem ağırlığı, 3 ile 1.000 arası. Büyük pazarda kazanılan puan daha değerli.
Aktarma ödülü — sunulan her geçerli bağlantı için azalan getirili puan: 1, ½, ¼, ⅛ …
Sıralama ödülü — kaç rakip geçtiğinize göre lookup tablosundan okunan puan.
λ · kaydırma — gereksiz tarife değişimini engelleyen ceza.

Uygun (feasible) ve uygun olmayan (infeasible)

Uygun çözüm: tüm kısıtları sağlayan herhangi bir çözüm. İyi olması gerekmez, sadece kuralları çiğnememesi yeter.

Infeasible: hiçbir çözümün tüm kısıtları aynı anda sağlayamaması. Bu bir hata değil, bir **bilgidir**: kısıtlarınız birbiriyle çelişiyor demektir.

Bu projede gerçekten yaşandı

Kısıt **B2** ("uygun olan her bağlantıyı sunmak zorundasın") ile **E1** ("gidiş-dönüş dengeli olmalı") çatıştı. Asimetrik pazarlarda — bir yönde çok, ters yönde az bağlantı olan pazarlarda — ikisi aynı anda sağlanamıyor ve model **infeasible** oluyordu.

Çözüm: B2 kapatıldı. Zaten amaç fonksiyonu sunmayı ödüllendirdiği için model kârlı bağlantıları kendiliğinden sunuyor. Bu, kodda `bisection` yöntemiyle doğrulanmış bir karar (`03_model.py:86–89`).

5.2 Modelleme teknikleri — asıl zanaat

Doğrusallık — neden bu kadar takıntılıyız?

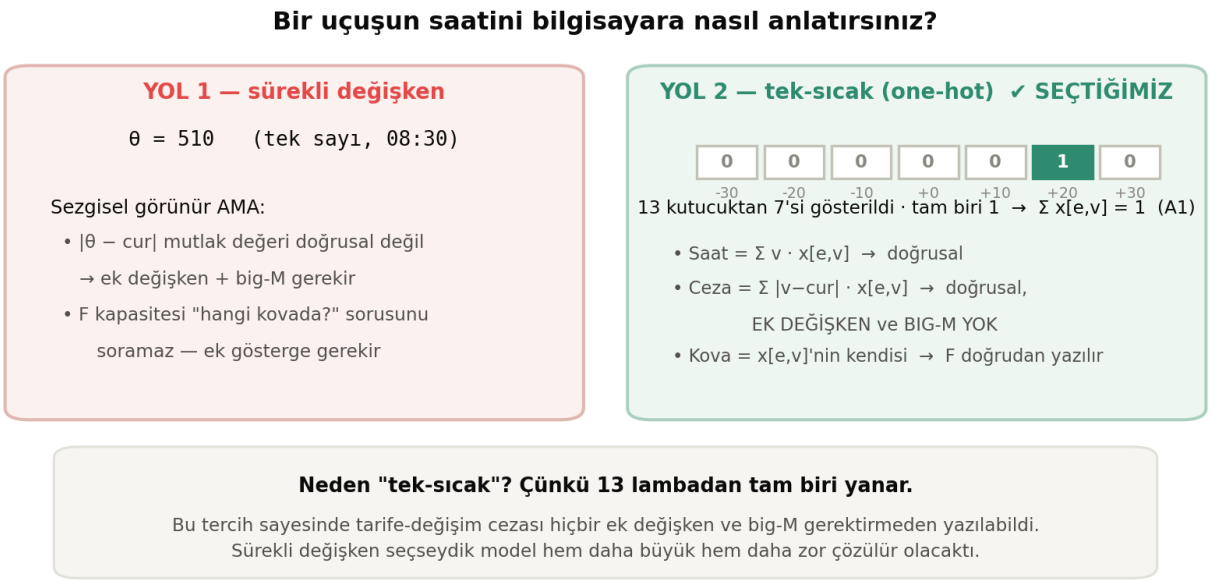
Doğrusal ifade nedir? Değişkenlerin yalnızca sabitlerle çarpılıp toplandığı ifade. `3x + 5y − 2z` doğrusaldır. `x·y` (iki değişkenin çarpımı), `x²`, `|x|`, `min(x,y)`, `eğer x>3 ise…` doğrusal **değildir**.

Neden önemli? Çünkü MIP çözücüleri **yalnızca doğrusal** ifadeleri çözebilir. Doğrusal olmayan bir şey yazmak istiyorsanız, onu doğrusal parçalardan yeniden inşa etmeniz gerekir. Modelleme zanaatının büyük kısmı budur.

İstediğimiz (doğrusal değil)	Nasıl doğrusallaştırdık?	Kısıt
"Eğer sunuluyorsa boşluk 60–300 olsun"	big-M numarası	B1
"1. bağlantı 1 puan, 2. bağlantı ½ puan…"	monoton `z` slot dizisi	C
"Lookup tablosundan puan oku"	`y` seçici değişkenleri	D
"|θ − mevcut saat| kadar ceza"	one-hot sayesinde `Σ|v−cur|·x` — **hiç numara gerekmedi**	P
"En kısa J'yi seç"	`u` seçici + iki yönlü big-M	E2

Tek-sıcak (one-hot) kodlama

Nedir? Bir seçimi tek bir sayıyla değil, **her seçenek için bir evet/hayır lambası** ile ifade etmek. Tam bir lamba yanar, gerisi söner.



Şekil 5.1 — Aynı bilgiyi iki farklı şekilde ifade etmek.

Bir uçuşun saatini iki şekilde yazabilirdik:

1. **Sürekli değişken:** `θ = 510` (yani 08:30). Tek sayı, sezgisel görünür.
2. **Tek-sıcak:** 13 tane 0/1 değişkeni; hangi kova seçildiyse o 1, diğerleri 0.

İkincisini seçtik. Nedeni tek kelimeyle: **ceza fonksiyonu.**

Kritik kazanç

Tarife-değişim cezası `|θ − mevcut saat|` mutlak değeri gerektirir. Mutlak değer doğrusal değildir; normalde ek değişken ve big-M ister.

Ama A1 kısıtı sayesinde her olayda **tam bir** `x` değişkeni 1'dir. O zaman:

|θ − cur| = Σ_v |v − cur| · x[olay, v]

Buradaki `|v − cur|` değerleri **sabittir** (aday saatler önceden belli). Yani ifade tamamen doğrusal — **sıfır ek değişken, sıfır big-M.**

İkinci kazanç: **F kapasitesi**. "Bu kovada kaç uçuş var?" sorusu, one-hot ile doğrudan `Σ x[olay, kova] ≤ 10` olarak yazılır. Sürekli değişkenle bu soruyu sormak için ayrıca gösterge değişkenleri kurmak gerekirdi.

big-M

Nedir? "Eğer şu koşul geçerliyse şu kural işlesin" mantığını doğrusal hale getiren bir numara. Adı, kullanılan büyük sabitten (M) gelir.



Şekil 5.2 — Aynı eşitsizlik, y'nin değerine göre devreye giriyor veya etkisizleşiyor.

Somut örnek. İstediğimiz kural: *bir bağlantı sunuluyorsa (y=1) boşluğu en az 60 dakika olmalı.* Bunu doğrudan yazamayız. Bunun yerine şunu yazarız:

> **boşluk ≥ 60 − Mlo · (1 − y)**
>
> **y = 1 ise:** sağ taraf `60 − Mlo·0 = 60` olur → kural **devreye girer**.
> **y = 0 ise:** sağ taraf `60 − Mlo` olur. Mlo yeterince büyük seçilirse bu sayı, boşluğun alabileceği en küçük değerden bile küçüktür → kural **otomatik sağlanır, yani etkisizleşir**.

Peki M ne kadar büyük olmalı? İşte asıl mesele bu.

M çok küçükse	M çok büyükse	Doğrusu
Kural y=0 iken bile bağlar → **model yanlış olur**	Model **doğru** ama çözücü çok yavaşlar — doğrusal gevşeme zayıflar, hiçbir dal budanamaz	Kuralı etkisizleştirecek **en küçük** M

> **Bizim yaptığımız**
>
> Genel bir "büyük sayı" (mesela 99999) kullanmak yerine, **her bağlantı için ayrı ayrı** hesapladık:
>
> ```
> Mlo = max(0, 60 − o bağlantının ulaşabileceği en küçük boşluk)
> Mhi = max(0, o bağlantının ulaşabileceği en büyük boşluk − 300)
> ```
>
> Aday saatler sınırlı olduğu için bu aralıklar önceden bilinebiliyor. Sonuç: mümkün olan en sıkı M değerleri → çözücü çok daha hızlı ilerliyor (`03_model.py:382-390`).

Seyrek (sparse) değişken üretimi

Nedir? İhtiyaç duyulmayan değişkeni **hiç yaratmamak**. Kulağa basit gelir ama büyük modellerde çözülebilirlik ile çözülemezlik arasındaki fark budur.

Örnek: teorik olarak her (bağlantı × rakip) çifti için bir "yendi mi?" değişkeni gerekir — 13.015 çift. Kod üç duruma ayırıyor:

Durum	Sayı	Ne yapılıyor?
Asla yenemez — izin verilen boşluk, ulaşılabilir en küçüğün altında	3.384 (%26)	Değişken **hiç yaratılmıyor**
Her zaman yener — pencere içindeki her boşlukta rakip geçiliyor	2.100 (%16)	`w` yerine doğrudan `y` kullanılıyor
Koşullu — boşluğa bağlı	7.531 (%58)	Gerçek `w` değişkeni + 2 kısıt

Yani çiftlerin **%42'si için hiç değişken yaratılmıyor.** Benzer eleme F kısıtında da var: kapasite kısıtı yalnız **bağlayıcı olabilecek** kovalara ekleniyor (1.395 kısıt).

5.3 Çözücü nasıl çalışıyor?

Doğrusal gevşeme (LP relaxation)

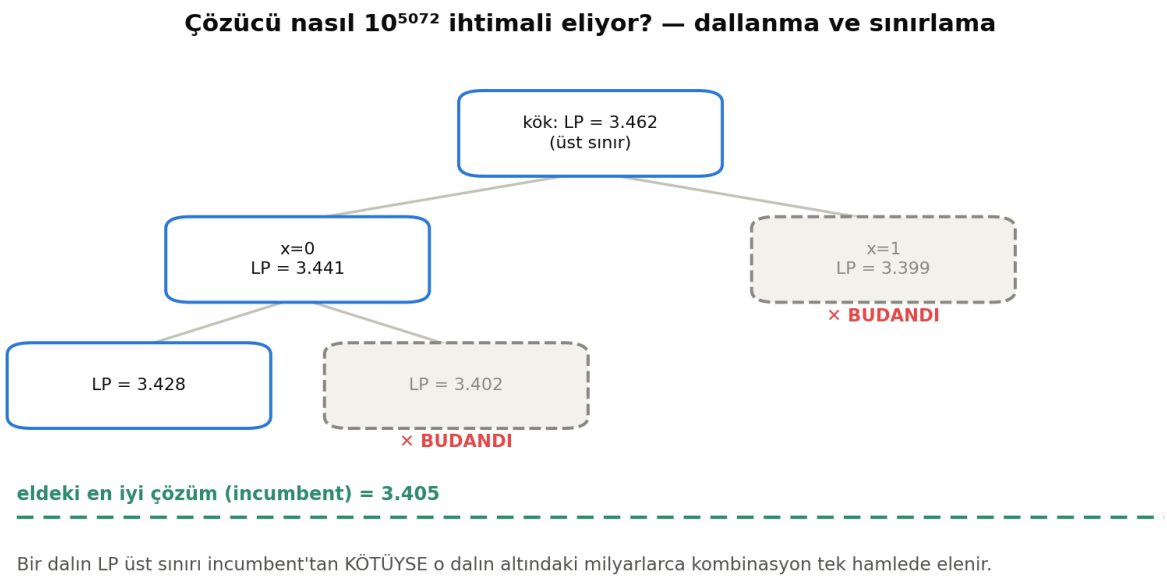
Nedir? Çözücünün ilk hamlesi şaşırtıcıdır: problemi **kolaylaştırır**. $x \in \{0,1\}$ şartını gevşetip $0 \le x \le 1$ yapar. Artık x kesirli olabilir.

Neden işe yarıyor? Gevşetilmiş problem çok hızlı çözülür ve verdiği sonuç, gerçek problemin sonucundan **daha iyi ya da eşittir** (daha az kısıt var). Yani bir **üst sınır** verir: "*gerçek cevap bundan yüksek olamaz.*"

> **Bir benzetme**
> Bir sınavda "tam not 100" bilgisi, kimsenin 100'ün üstünde alamayacağını söyler. Gevşeme de öyle: gerçek optimumun tavanını verir. Bu tavan olmadan hiçbir dal budanamaz.

Dallanma ve sınırlama (branch and bound)

Nedir? MIP çözücülerinin ana algoritması. Adım adım:



Şekil 5.3 — Bir dalın üst sınırı elimizdekinden kötüyse, o dal komple budanır.

1. Gevşeme çözülür. `x = 0,63` gibi kesirli bir değer çıkarsa problem ikiye **dallanır**: bir dalda `x = 0`, diğerinde `x = 1`.
2. Her dalda gevşeme yeniden çözülür ve o dalın **üst sınırı** bulunur.

3. Elde bir uygun çözüm (**incumbent**) varsa değeri bilinir — örneğin 3.405.000.
4. Bir dalın üst sınırı 3.399.000 ise, o dalın altında 3.405.000'i geçen bir çözüm **olamaz** → dal komple **budanır**.
5. Böylece $10^{5.072}$ ihtimalin ezici çoğunluğu **hiç ziyaret edilmeden** elenir.

Incumbent (eldeki en iyi çözüm)

Nedir? Şu ana kadar bulunmuş, tüm kısıtları sağlayan en iyi çözüm. Türkçede "mevcut en iyi" diyebiliriz.

Neden kritik? Budama ancak elde bir karşılaştırma noktası varken çalışır. Incumbent yoksa hiçbir dal budanamaz ve çözücü ağaçta boğulur.

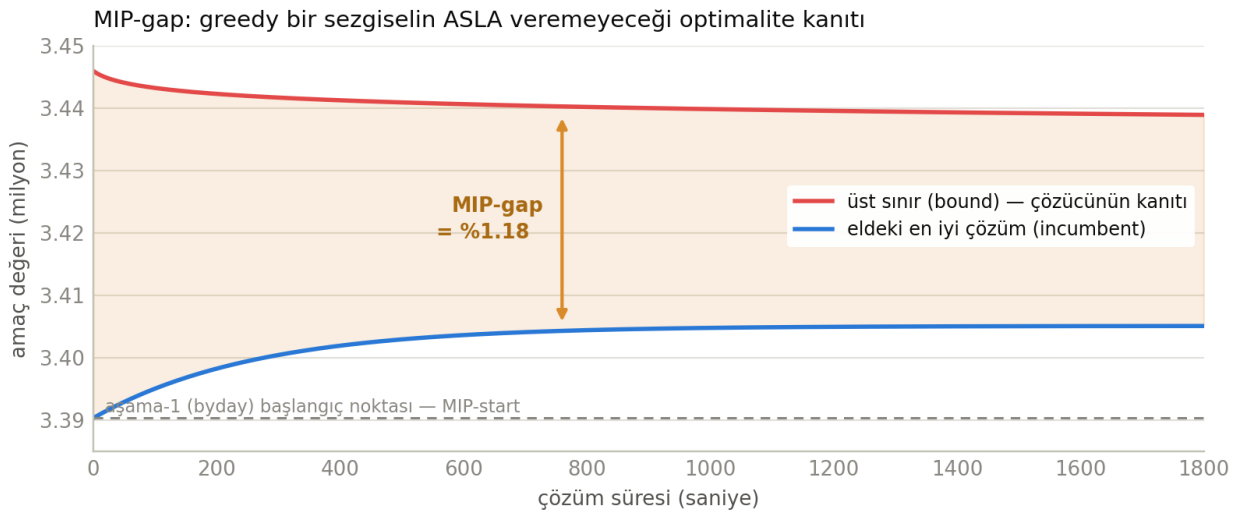
> **Bu projedeki en büyük teknik sorun tam da buydu**
>
> Tam-hafta modelinde (~100.566 ikili) HiGHS makul sürede **tek bir incumbent bile bulamıyordu**. Çözüm bulunamayınca budama yok, budama yoksa ilerleme yok.
>
> İşte `08_fullweek_polish.py` bu yüzden var: byday'in bulduğu çözümü hazır bir incumbent olarak çözücünün eline veriyoruz.

Üst sınır (bound) ve MIP-gap

Üst sınır: çözücünün kanıtladığı tavan — "gerçek optimum bundan yüksek olamaz".

MIP-gap: eldeki en iyi çözüm ile bu tavan arasındaki bağıl fark.

> **gap = |üst sınır − eldeki en iyi| / |eldeki en iyi|**
>
> Bizim teslim koşumuzda **%1,18**. Türkçesi:
>
> *"Bulduğum çözüm, teorik olarak mümkün olan en iyinin en fazla %1,18 altında."*



Şekil 5.4 — İki sınır birbirine yaklaşır; aradaki fark gap'tir.

> **Neden bu, projenin en değerli çıktısı?**
>
> Elle yazılmış hiçbir açgözlü algoritma bu cümleyi kuramaz. Greedy bir yöntem size bir tarife verir — ama o tarifenin optimale yakın mı yoksa çok mu uzak olduğu konusunda **hiçbir bilgi** vermez. Gap, "iyi bir çözüm buldum" ile "bulduğum çözümün ne kadar iyi olduğunu kanıtlayabiliyorum" arasındaki farktır.

Kesme düzlemleri (cutting planes)

Nedir? Çözücünün kendi ürettiği ek eşitsizlikler. Kesirli çözümü dışarıda bırakırlar ama hiçbir geçerli tamsayı çözümü kesmezler. Sonuç: gevşeme sıkılaşır, üst sınır düşer, budama hızlanır.

Bunlar sizin yazmadığınız kısıtlardır — çözücü otomatik türetir.

Sezgisel (heuristic)

Nedir? "En iyi"yi garanti etmeyen ama hızlıca **iyi** bir çözüm bulan yöntem. Çözücüler bunları incumbent'ı erken bulmak için kullanır (RINS, local branching, feasibility pump).

Bu projede sezgisel bütçesi bilinçli olarak artırıldı: `mip_heuristic_effort` varsayılan 0,05 iken **0,3** yapıldı. Gerekçe: warm-start'lı büyük MIP'te iyileştirmeler çoğunlukla incumbent çevresinde çalışan sezgisellerden gelir (`08_fullweek_polish.py:412-414`).

Determinizm

Nedir? Aynı girdiyle her zaman aynı çıktıyı üretme özelliği.

Neden zor? Çok çekirdekli MIP çözümü doğası gereği deterministik değildir: iş parçacıkları düğümleri farklı sırayla işler, farklı çözümler dönebilir.

> **Bizim tercihimiz**
>
> `{"threads": 1, "random_seed": 0}` — tek iş parçacığı, sabit tohum. Bu **hızdan feragat** demektir. Ama yarışma teslimi için "jüri kendi makinesinde koşturunca aynı sayıyı görmeli" kısıtı hız kaygısının önüne kondu (`03_model.py:922`).

5.4 Çözüm stratejisi terimleri

Warm-start (MIP-start)

Nedir? Çözücüye sıfırdan başlamak yerine "*işte elimde çalışan bir çözüm, buradan devam et*" demek.

Nasıl çalışır? Elinizdeki çözümün **tüm** değişken değerlerini çözücüye yazarsınız. Çözücü bunu incumbent kabul eder ve o noktadan **iyileştirmeye** başlar; ayrıca üst sınırı daraltarak gap üretir.

	Warm-start'sız	**Warm-start'lı**
Başlangıç	Sıfırdan çözüm arar	Elde 3.390.287 puanlık çözüm var
Bu modelde sonuç	**Hiç incumbent bulamıyor**	İlk saniyeden itibaren budama yapabiliyor
Elde edilen	—	**+14.752** puan ve **%1,18** global gap

> **Kritik ayrıntı — başlangıç noktası KUSURSUZ olmalı**
>
> HiGHS, verilen başlangıç noktasını uygunluk açısından denetler. **Tek bir kısıt ihlali** varsa noktayı tümüyle reddeder ve sıfırdan başlar — yani warm-start'ın tüm anlamı kaybolur.
>
> Bu yüzden `08_fullweek_polish.py`'de `y` değişkenleri "uygunluktan" değil, byday'in **gerçek sunum listesinden** okunuyor. byday, E1 dengesi için bazı uygun bağlantıları bilinçli sunmuyor; hepsini y=1 yapmak E1'i ihlal eden bir başlangıç üretirdi. Ayrıca bir **E1 onarım** döngüsü de var.

Matheuristik

Nedir? Kesin yöntem (matematiksel programlama) ile sezgiseli birleştiren melez yaklaşım. Bizimki *fix-and-optimize / warm-start* ailesine girer:

1. **Sezgisel adım:** haftayı günlere böl, her günü ayrı çöz (hızlı, garantili çözüm — ama küresel optimallik iddiası yok).
2. **Kesin adım:** bu çözümü tam-hafta modeline başlangıç ver, çözücü küresel olarak iyileştirsin ve gap'i kanıtlasın.

Ayrıştırma (decomposition)

Nedir? Çözülemeyecek kadar büyük bir problemi çözülebilir parçalara bölmek.

Bizde bölme ekseni **gün**: 7 günü ayrı ayrı çözüyoruz. Ama günleri birbirine bağlayan iki şey var ve ikisi de ayrıca ele alınıyor:

- **G düzenliliği** — aynı uçuş her gün benzer saatte olmalı. Çözüm: önceki günlerde yerleşen saat bir "bant" olarak sonraki güne aktarılıyor (zincirleme).
- **A2 gece-aşan rotasyon** — gün h'nin kalkışı ile gün h+1'in dönüşü aynı modelde değil. Çözüm: `compute_overnight_guards` ile statik slack bölüşümü.

Açgözlü (greedy) yöntem

Nedir? Her adımda o an en iyi görünen kararı alan, geri dönüp düzeltmeyen yöntem. Hızlıdır, ama küresel en iyiyi bulacağının garantisi yoktur.

byday ayrıştırması bir greedy zincirdir: 3. günde alınan bir karar 6. gün için daha iyi bir seçeneği kapatmış olabilir ve geri alma imkânı yoktur. **Aşama-2 tam olarak bunu telafi ediyor.**

Dejenerasyon ve λ cezası

Dejenerasyon nedir? Birden fazla çözümün **tamamen aynı** amaç değerini vermesi. Çözücü aralarından rastgele birini seçer.

> **Bu neden bir sorun?**
>
> λ = 0 iken (ceza yok) çözücü, amaç değerini **hiç değiştirmeyen** kaydırmaları da yapıyordu — çünkü yapmamak için bir nedeni yok. Sonuç: uçuşların **~%90'ı** boşuna kaydırılıyordu.
>
> Havayolu için bu, sıfır kazanç karşılığı devasa bir operasyonel maliyettir: yer hizmetleri, mürettebat planı, yolcu bilgilendirme, sistem güncellemeleri.

λ cezası bu simetriyi kırar: amaçtan `λ · toplam kaydırma` düşülür. Artık bir kaydırmanın hayatta kalması için **gerçek kazanç** getirmesi gerekir. Sonuç: uçuşların %90'ı yerine yalnız **%23'ü** kaydırıldı, kazanç ise arttı.

λ = 0,3 değeri tahminle değil **taramayla** seçildi: λ ∈ [0,1 – 1,5] aralığında kazanç düz bir plato çiziyor; 0,3 bu platonun dizi (*knee*) noktası.

Pareto ödünleşimi ve "knee" noktası

Pareto ödünleşimi: iki iyi şeyin aynı anda en üst düzeye çıkarılamaması. Burada: *daha çok kazanç ↔ daha az tarife değişikliği*.

Knee (diz) noktası: eğrinin kırıldığı yer — bir birim daha kazanç için ödemeniz gereken bedelin hızla arttığı nokta. Rasyonel seçim genellikle buradadır.

Duyarlılık analizi

Nedir? Parametreleri oynatıp modelin tepkisine bakmak. Amaç sonucu iyileştirmek değil, sonucun **ne kadar sağlam** olduğunu göstermek.

Beklenen davranış: **sürekli ve yönü açıklanabilir** tepki. Küçük bir parametre değişikliğinde sonuç sıçrıyorsa modelde ya kırılganlık ya hata vardır. `07_sensitivity.py` üç parametrede 11 koşu yapıyor (λ, Wdev, τ) ve **teslim edilen çözüme dokunmuyor**.

5.5 Problem ve havacılık terimleri

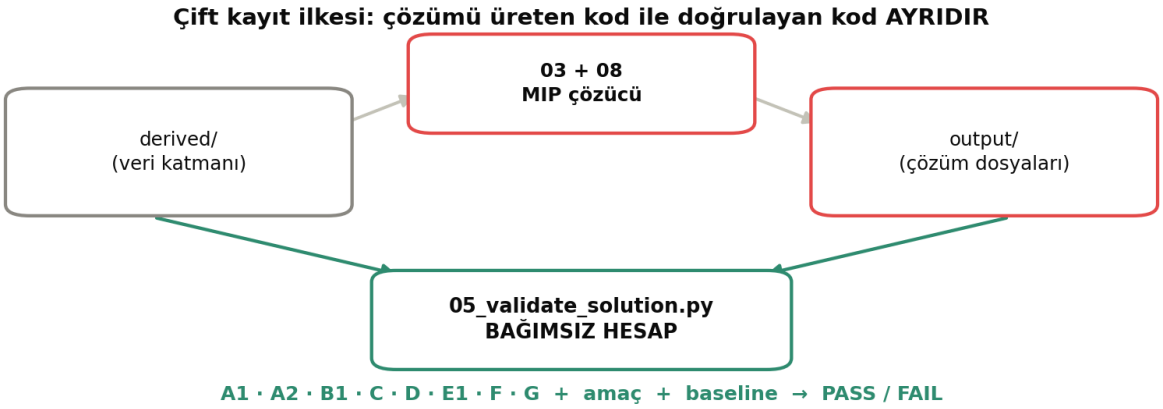
Terim	Nedir?	Bu projede
O–D pazarı	Origin–Destination: yönlü kalkış–varış çifti	1.408 yönlü pazar. Örn. `AGP-KCM` ile `KCM-AGP` **farklı** pazarlardır
Itinerary / bağlantı	Bir aktarmalı yolculuk seçeneği	TK'nınki 7.850 satır, rakiplerinki 49.467 satır
Hub / aktarma merkezi	Yolcuların uçuş değiştirdiği merkez havalimanı	İstanbul (IST)
Bacak (leg)	Tek bir uçuş — aktarmanın yarısı	1. bacak: kalkış→IST · 2. bacak: IST→varış
Boşluk (gap)	IST'te bekleme süresi	Brief'in penceresi: 60–300 dakika
J	Toplam yolculuk süresi = 1. bacak + boşluk + 2. bacak	Rakiple kıyaslanan **tek** büyüklük. En uzunu 35 saat
legsum	İki bacağın uçuş süreleri toplamı	**Sabittir** — model değiştiremez; yalnız boşluk değişir
ρ (rho)	Pazarın gelir/önem ağırlığı	3 – 1.000 arası. Amaç fonksiyonundaki çarpan
N_od	Pazardaki rakip sayısı	**Taşıyıcı bazlı** sayılıyor (en fazla 17, ödül tablosu 40'a kadar destekliyor)
T_comp	Bir rakip taşıyıcının o pazardaki en iyi J değeri	`J ≤ T_comp` ise o rakibi yendik demektir
b_od	Pazarın başlangıç sırası	Mevcut tarifeye göre hesaplanır; sıralama ödülü b'den s'ye geçişin ödülüdür
s (son sıra)	clip(N − yenilen, 1, N)	Küçük s = daha iyi sıralama
τ (tau) / MCT	Minimum yer süresi — uçağın varış havalimanında geçirmesi gereken en az süre	45 dakika
Blok süresi	Bir bacağın uçuş süresi (T_OB gidiş, T_IB dönüş)	`ElapsedTime1/2`'den türetiliyor; A2 kısıtının girdisi
Rotasyon	Aynı uçağın yaptığı gidiş–dönüş turu	`Flight Pairs.xlsx`'ten okunuyor; 330 çift
Kova (bucket) / slot	10 dakikalık zaman dilimi	Hem zaman ızgarası hem F kapasitesinin sayım birimi
Baseline	Mevcut tarifenin (hiç dokunulmamış) amaç değeri	3.251.527 — tüm iyileşme buna göre ölçülüyor

5.6 Kalite ve doğrulama terimleri

Çift kayıt (double-entry)

Nereden geliyor? Muhasebeden. Her işlem iki kez, iki farklı yere kaydedilir; ikisi tutmuyorsa hata vardır.

Bizde: çözümü üreten kod (`03` + `08`) ile doğrulayan kod (`05`) **ayrıdır**. 05 ne Pyomo import ediyor ne de modele dokunuyor; sabitleri kendi dosyasında yeniden yazıyor. Böylece modeldeki bir hata, doğrulamada aynı hatayla örtülemiyor.



Tek bir FAIL → çıkış kodu 1 → run_all.py koşuyu durdurur.

05, modelin içinden HİÇ geçmez; sabitleri (Wdev, τ, Xdev, cap) kendi dosyasında yeniden tanımlar.
Böylece modeldeki bir hata, doğrulamada aynı hatayla örtülemez.

Şekil 5.5 — Üreten kod ile doğrulayan kod ayrıdır.

Fazla-beyan / eksik-beyan asimetrisi

	Ne demek?	Nasıl işleniyor?
Fazla-beyan	Dosyada yazandan az rakip gerçekten yenilmiş → amaç değeri **şişirilmiş**	**HATA** — doğrulama FAIL verir
Eksik-beyan	Gerçekte yenilmiş ama çözücü göstergeyi bağlamamış → amaç değeri olduğundan küçük görünüyor	**NOT** — hata değil, ihtiyatlılık

Bu asimetri bilinçlidir: **kendi lehimize olan sapmalar hata sayılıyor, aleyhimize olanlar sayılmıyor.** Ölçüm kendine karşı kurulmuş.

Eşleşme artefaktı (A2)

Nedir? Rotasyon eşleşmesinin, mevcut yayınlanan tarifenin kendisinin de sağlamadığı bir tur öngörmesi. Yani o gün o çift muhtemelen tek uçakla dönülmüyor — veri yorumunun sınırı.

Kod bunu **ihlal saymıyor** ama **sayıyor ve raporluyor**. Gerçek hata olan "YENİ ihlal" (mevcut sağlıyordu, optimizasyon bozdu) ayrı sayılıyor ve **0** olmak zorunda.

Kalite kapısı

Nedir? Boru hattının içine gömülmüş, geçilemezse akışı durduran kontrol.

Bizde: `05_validate_solution.py` bir tane bile FAIL bulursa **çıkış kodu 1** döndürüyor; `run_all.py` bunu görüp koşuyu durduruyor. Yani `output/` klasöründe bir sonuç varsa ve uçtan uca koşu başarıyla bittiyse, o sonuç **kesinlikle** 12 kontrolden geçmiştir. Doğrulanmamış bir çözüm için figür bile üretilmez.

6 Jüri Soru Bankası — 45 soru, açıklamalı cevap

Sorular altı gruba ayrıldı. Her cevap **kısa bir özet cümleyle** başlıyor; ardından gerekçe ve sayısal dayanak geliyor. Sunumda önce özet cümleyi söyleyin, jüri derinleşmek isterse gerisini anlatın.

A · Problem anlayışı

S1 Problemi kendi cümlelerinizle anlatır mısınız?

THY'nin yolcularının büyük kısmı doğrudan uçmuyor: bir uçuşla İstanbul'a geliyor, bekliyor, ikinci uçuşla gideceği şehre gidiyor. Yolcu için önemli olan **kapıdan kapıya toplam süre**. Aynı yolcuyu rakipler kendi merkezleri üzerinden taşımak istiyor — kim daha hızlıysa tercih ediliyor.

Bizim elimizdeki tek kaldıraç **İstanbul'daki iniş ve kalkış saatleri**. Bu saatleri, tüm operasyonel kuralları koruyarak, gelir-ağırlıklı rekabetçilik puanını en yükseğe çıkaracak şekilde yeniden düzenledik.

S2 Neden sadece saatleri değiştiriyorsunuz? Uçuş ekleseniz daha çok kazanmaz mıydınız?

Kazanırdık — ama o başka bir problem. Uçuş eklemek filo, mürettebat, slot ve yatırım kararıdır; brief bize tarife saatlerini soruyor. Ayrıca **dar karar alanı bir avantaj**: elde ettiğimiz %4,72'lik kazancın kaynağı belirsiz değil — 1.067 uçuş hareketinin ortalama 19 dakika kaydırılması. Uçuş eklemek ya da fiyat oynatmak gibi başka kaldıraçlar karışmadığı için kazanç temiz ve savunulabilir.

S3 ±60 dakikalık pencere ve 10 dakikalık ızgara nereden geldi?

İkisi de bizim modelleme tercihimiz — brief bir sınır koymuyor.

- **±60 dakika:** sınırsız bırakılsaydı model bir uçuşu sabahtan akşama taşıyabilirdi. Kâğıtta iyi, gerçekte uygulanamaz. Pencere "iyileştirme yap ama tarifeyi tanınmaz hâle getirme" demenin sayısal karşılığı.
- **10 dakika:** havayolu tarifeleri zaten 5–10 dakikalık dilimlerde yayınlanır. Ayrıca hub kapasitesi de 10 dakikalık kovalarda sayılıyor — ikisinin aynı birim olması kısıtların birbirine çevrilebilmesini sağlıyor.
- Duyarlılık analizinde her ikisi de sınandı (`07_sensitivity.py`); Wdev=30 ve Wdev=90 varyantları raporlandı.

S4 Bağlantı penceresi neden 60–300 dakika?

Bu brief'ten geliyor, bizim seçimimiz değil. Mantığı: 60 dakikadan kısa bir aktarmaya yolcu yetişemez (bagaj, pasaport, kapı değişimi); 300 dakikadan uzun bekleme ise satılabilir bir ürün değildir — yolcu başka havayolunu tercih eder.

S5 Amaç fonksiyonunuzu açıklar mısınız?

Amaç = Σ_pazar ρ · (aktarma ödülü + sıralama ödülü) − λ · Σ|kaydırma|

ρ — pazarın gelir ağırlığı (3–1.000). Büyük pazardaki kazanç daha değerli.
Aktarma ödülü — sunulan her geçerli bağlantı için azalan getirili puan (1, ½, ¼ …). Gerçek hayatta ilk aktarma

seçeneği çok değerlidir, beşincisi neredeyse hiç.
Sıralama ödülü — kaç rakip geçtiğimize göre organizatörün verdiği lookup tablosundan okunur.
λ · kaydırma — gereksiz tarife değişimini engelleyen ceza (λ = 0,3).

Önemli not: raporladığımız tüm sonuçlar **cezasız (saf) rekabetçilik** üzerindendir. λ yalnızca hangi kaydırmaların yapılacağını süzer; puanı ölçmez.

B · Model ve formülasyon

S6 Neden MIP? Neden makine öğrenmesi kullanmadınız?

Çünkü bu bir **tahmin** problemi değil, bir **karar** problemi. Makine öğrenmesi geçmişten öğrenip gelecek tahmin eder; burada tahmin edilecek bir şey yok — kısıtlar altında en iyi kararı arıyoruz.

	Sinir ağı olsaydı	MIP ile
Kısıtlar	Garanti edilemez — ihlal edebilir	**Matematiksel olarak garanti**
"En iyi mi?"	Cevaplanamaz	**%1,18 gap ile kanıtlanır**
Açıklanabilirlik	Kara kutu	Her kısıt ve terim tek tek okunabilir
Veri ihtiyacı	Binlerce etiketli örnek	Yok — kurallar yeterli

S7 Kaç değişken ve kaç kısıt var?

100.566 ikili değişken, 70.359 kısıt. Dökümü: `x` (uçuş saati atamaları) 59.189 · `γ` (yenilen rakip sayısı seçicileri) 11.690 · `y` (bağlantı sunuluyor mu) 7.577 · `z` (ödül slotları) 7.577 · `w` (rakip yeniliyor mu) 7.531 · `β` (rakip yenildi mi) 7.002.

S8 Karar değişkeniniz tam olarak ne? Neden bağlantı bazlı değil?

Karar birimimiz **olay**: `(uçuş numarası, rol, gün)`. Rol = IST'e varış veya IST'ten kalkış.

Neden bağlantı bazlı değil — bu kritik bir tasarım kararı

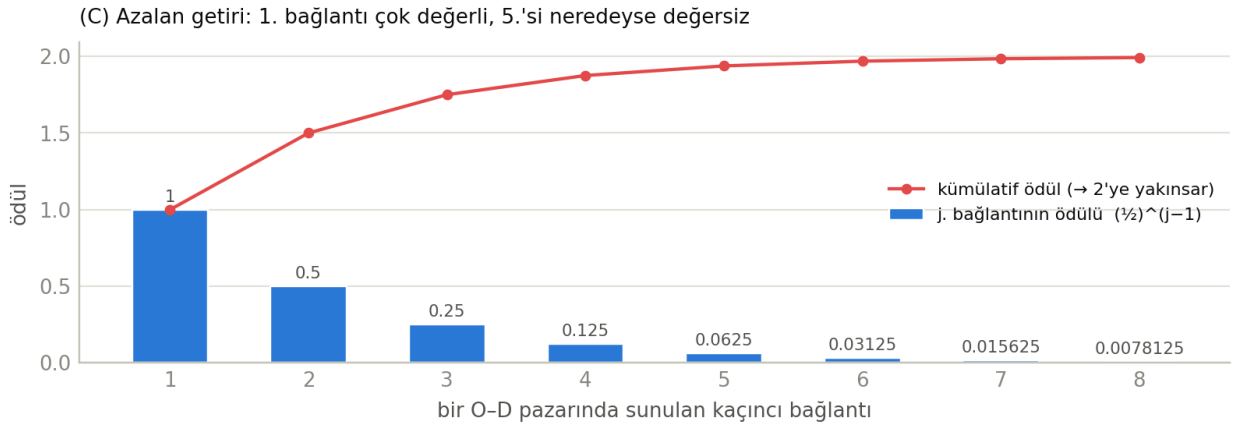
Bir uçuş **20 farklı bağlantıda** yer alabilir: IST'e gelen bir uçak, oradan kalkan 20 uçuşla 20 bağlantı oluşturur. Bağlantı başına karar verseydik aynı uçağa 20 farklı varış saati atanabilirdi — fiziksel olarak imkânsız bir tarife.

Olay soyutlaması bu tutarsızlığı **yapısal olarak imkânsız** kılıyor: aynı anahtar → aynı karar.

S9 Azalan getiri ödülü neden (½)^(j−1)?

Gerçek hayatta bir pazarda **ilk** aktarma seçeneği çok değerlidir; ikincisi biraz daha az; beşincisi neredeyse hiç. Yolcu zaten bir tane seçecek.

Geometrik dizi seçilmesinin iki nedeni var: (i) hızlı ama pürüzsüz azalıyor, (ii) toplamı 2'ye yakınsıyor — yani tek bir pazarın puanı sınırsız büyüyemiyor, model tek pazara odaklanıp diğerlerini ihmal edemiyor.



Şekil 6.1 — Ödül dizisi ve kümülatif toplam.

S10 big-M nedir? M değerlerini nasıl seçtiniz?

big-M, "*eğer şu koşul geçerliyse şu kural işlesin*" mantığını doğrusal hale getiren bir tekniktir. Örnek: "bağlantı sunuluyorsa (y=1) boşluk ≥ 60" kuralını `boşluk ≥ 60 − Mlo·(1−y)` olarak yazarız. y=1 iken kural devreye girer, y=0 iken sağ taraf çok küçülür ve kural etkisizleşir.

M seçimi neden kritik?

Çok küçük M: kural y=0 iken bile bağlar → model **yanlış** olur.
Çok büyük M: model doğru ama doğrusal gevşeme zayıflar, hiçbir dal budanamaz, çözücü ağaçta kaybolur.

Biz genel bir sabit yerine **her bağlantı için ayrı ayrı** hesapladık: `Mlo = max(0, 60 − o bağlantının ulaşabileceği en küçük boşluk)`. Aday saatler sınırlı olduğu için bu aralık önceden bilinebiliyor — sonuç mümkün olan en sıkı M.

S11 Neden tek-sıcak (one-hot) kodlama? Sürekli bir saat değişkeni daha basit olmaz mıydı?

Görünüşte evet, pratikte hayır. Tek-sıcak kodlama iki büyük kazanç sağladı:

1. **Tarife-değişim cezası tam doğrusal oldu.** `|θ − mevcut saat|` mutlak değeri normalde ek değişken ve big-M ister. Ama her olayda tam bir `x` = 1 olduğu için `|θ−cur| = Σ|v−cur|·x[olay,v]` yazılabiliyor — katsayılar sabit, **sıfır ek değişken, sıfır big-M**.
2. **F kapasitesi doğrudan yazılabildi.** "Bu kovada kaç uçuş var?" sorusu `Σ x[olay, kova] ≤ 10` oluyor. Sürekli değişkenle bu soruyu sormak için ayrıca gösterge değişkenleri kurmak gerekirdi.

S12 B2 kısıtını neden kapattınız? Model eksik kalmıyor mu?

B2 "uygun olan her bağlantıyı sunmak zorundasın" der. Açık olduğunda **E1 yön dengesiyle çatışıyor**: bir yönde çok, ters yönde az bağlantı olan asimetrik pazarlarda ikisi aynı anda sağlanamıyor ve model **infeasible** oluyor — yani hiç çözüm alamıyoruz.

Model eksik kalmıyor çünkü **amaç fonksiyonu zaten sunmayı ödüllendiriyor** (aktarma ödülü). Model kârlı olan her bağlantıyı kendiliğinden sunuyor. B2 açık olsaydı hiçbir sonucumuz olmazdı.

S13 E2 kısıtını tam ölçekte kapattınız — bu modeli zayıflatmıyor mu?

E2, gidiş ve dönüş yönlerinin en iyi yolculuk sürelerinin 30 dakikadan fazla ayrılmamasını ister. **Küçük alt-örnekte açık ve doğrulanmış** durumda; yalnız tam ölçekte kapalı.

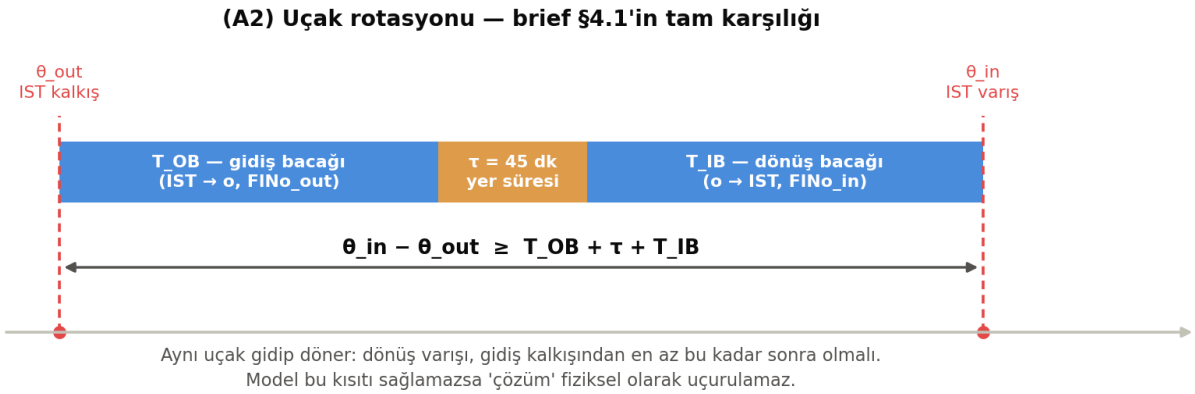
Gerekçe ölçülmüş: E2'nin "minimum seçimi" yapısı bağlantı başına iki big-M kısıtı ve bir ek ikili değişken getiriyor; doğrusal gevşemesi o kadar zayıflıyor ki HiGHS tam veride **tek bir uygun çözüm bile** bulamıyor. Tanı yöntemi kodda yazılı: çekirdek + yalnız E2 çözülemiyor, çekirdek + A2/G/F/E1 hepsi hızlı.

En önemlisi: bu karar **gizlenmedi**. `output/results_summary.md`'nin notlar bölümünde açıkça ilan ediliyor.

S14 A2 uçak rotasyon kısıtını nasıl kurdunuz?

Temel kural: **dönüş varışı ≥ gidiş kalkışı + T_OB + τ + T_IB** (gidiş uçuş süresi + 45 dk yer süresi + dönüş uçuş süresi). Aynı uçak gidip döneceği için fiziksel zorunluluk.

İncelik `sep` değerinin nasıl seçildiğinde: `sep = min(brief-exact, mevcut tur)`. Bu iki riski aynı anda kapatıyor — gerçek aynı-gün turlar asla brief minimumunun altına sıkışmıyor, gece-aşan çiftlerde ise mevcut desen bozulmuyor.



Şekil 6.2 — Minimum tur süresi.

S15 Gün ayrıştırması yaptınız — gece-aşan turlar nasıl korunuyor?

Bu, projenin en zarif teknik parçası. Gün h'nin kalkışı ile gün h+1'in dönüşü farklı MIP'lerde çözülüyor, yani model bu ikisini bağlayamaz.

Çözüm: mevcut tarifedeki boşluk payı (slack) **statik olarak ikiye bölünüyor** ve iki tek-taraflı sınır konuyor — bugünün kalkışı en fazla `slack/2` geç kayabilir, bugünün dönüş varışı en fazla `slack/2` erken kayabilir. İkisi birlikte minimum tur eşitsizliğini **yapısal olarak** garanti ediyor: iki gün birbirini hiç görmese bile ihlal matematiksel olarak imkânsız.

S16 Hub kapasitesi 10/15 nereden geliyor? Gerçekten bağlayıcı mı?

Brief'ten: her 10 dakikalık kovada en fazla 10 kalkış, 15 varış.

Kritik veri bulgusu

Mevcut tarifedeki kova aşımlarının **tamamı** ayarlanabilir uçuşların kümelenmesinden geliyor; **sabit uçuşlar hiçbir kovada baz kapasiteyi aşmıyor** (kalkışta en fazla 4 ≤ 10, varışta 5 ≤ 15).

Bu bulgu sayesinde brief'in **gerçek** 10/15 değerini uygulayabildik. Aksi hâlde `max(baz, mevcut)` gibi gevşetilmiş bir kapasite kullanmak gerekirdi ki bu F'yi fiilen etkisiz bir kısıta çevirirdi.

Ayrıca modellenmeyen (ρ'suz) pazarların uçuşları da kovalarda **sabit doluluk** olarak sayılıyor — fiziksel kapasite tüm TK uçuşlarıyla korunuyor.

S17 Tarife düzenliliği (G) neden gerekli?

Yolcu "her gün 08:30 uçuşu" bekler. Aynı uçuş numarası bir gün 07:00, ertesi gün 09:00 kalkarsa tarife satılabilirliğini kaybeder; ayrıca mürettebat ve yer hizmetleri planlaması bozulur. G, aynı uçuşun saat-içi zamanını günler arası ±15 dakikalık bir bantta tutuyor.

S18 E1 yön dengesi neden var? Kârlı yönü şişirmek daha iyi olmaz mı?

Kâğıtta daha iyi olur, gerçekte olmaz. Yolcu **gidiş-dönüş** ister; tek yönü şişirip dönüşü ihmal eden bir tarife satılabilir bir ürün değildir. E1, `|n_od − n_do| ≤ 0,20 × (n_od + n_do)` ile bu gerçekliği modele sokuyor.

Bedeli var: model 611 uygun bağlantıyı sunmuyor, bunun bir kısmı E1 yüzünden. Bilinçli bir ticari tercih.

S19 Modelin en zayıf yanı ne?

Dürüst cevap: **E2'nin tam ölçekte kapalı olması.** Yönler arası süre dengesi şu an denetlenmiyor; yalnız bağlantı **sayısı** dengesi (E1) uygulanıyor. Küçük örnekte E2 açık ve doğrulanmış ama tam veride çözülebilirlik engeli var.

İkinci zayıflık: **823 uçuş olayı donduruldu** çünkü rotasyon partnerleri veride yok. Bu bilgi sağlanırsa daha iyi bir sonuç elde edilebilir.

S20 Neden 823 uçuşu kımıldatmadınız?

176 TK uçuş numarasının `Flight Pairs.xlsx`'te karşılığı yok. Yani o uçuşu hangi uçağın yaptığını, o uçağın sonra nereye gittiğini bilmiyoruz.

Kaydırabilirdik — puan artardı. Ama fiziksel olarak uygulanamayan bir tarife üretmiş olabilirdik ve **bunu kontrol edecek veri yok**. Projenin kuralı: *doğrulayamadığın şeye dokunma.*

S21 Kısıtlardan hangisi en çok bağlıyor?

Kısıt sayısına göre **B1** (15.154) ve **G** (4.503) en kalabalık olanlar. Ama gerçekte en çok bağlayan **E1**: 611 uygun bağlantının sunulmamasının başlıca nedeni yön dengesi. **F** ise mevcut tarifedeki kümelenmeyi dağıtmaya zorluyor.

S22 Ödül tablosunun (ranking lookup) mantığını anladınız mı?

Evet ve iç tutarlılığını da doğruladık: her N için tam N×N satır var, `İlk Rank > Rakip Sayısı` olan tutarsız satır yok, sıralamanın kötüleştiği durumlar (10.660 satır) tabloda tanımlı — yani tablo cezayı da içeriyor.

Kritik yorum kararımız: sıralama ödülü bir **iyileştirme** ödülü. Tablonun köşegeni (b == s) "değişim yok" nötr referansı. Bu yüzden `b_od`'yi `s` ile **birebir aynı tabandan** hesapladık; aksi hâlde mevcut tarife hiçbir şey yapmadan ödül kazanırdı.

C · Çözüm ve çözücü

S23 Neden HiGHS? Gurobi daha hızlı değil mi?

Gurobi daha hızlı, doğru. Ama HiGHS'i seçmemizin nedeni hız değil **doğrulanabilirlik**: HiGHS açık kaynak ve lisans gerektirmiyor. Jüri `pip install -r requirements.txt` deyip kodu kendi makinesinde koşturabilir. Gurobi olsaydı lisans anahtarı gerekirdi ve sonuç bağımsız olarak doğrulanamazdı.

Ayrıca Pyomo kullandığımız için çözücü değiştirmek **tek satır**: `CFG["solver"]`. Model kodunun hiçbir satırı değişmiyor.

S24 `threads = 1` performansı düşürmüyor mu?

Evet, düşürüyor — ve bu bilinçli. Çok çekirdekli MIP çözümü **deterministik değildir**: iş parçacıkları düğümleri farklı sırayla işler, aynı girdiyle farklı çözümler dönebilir. Yarışma teslimi için "jüri koştuğunda aynı sayıyı görmeli" kısıtını hız kaygısının önüne koyduk (`threads=1`, `random_seed=0`).

S25 Neden haftayı tek seferde çözmediniz?

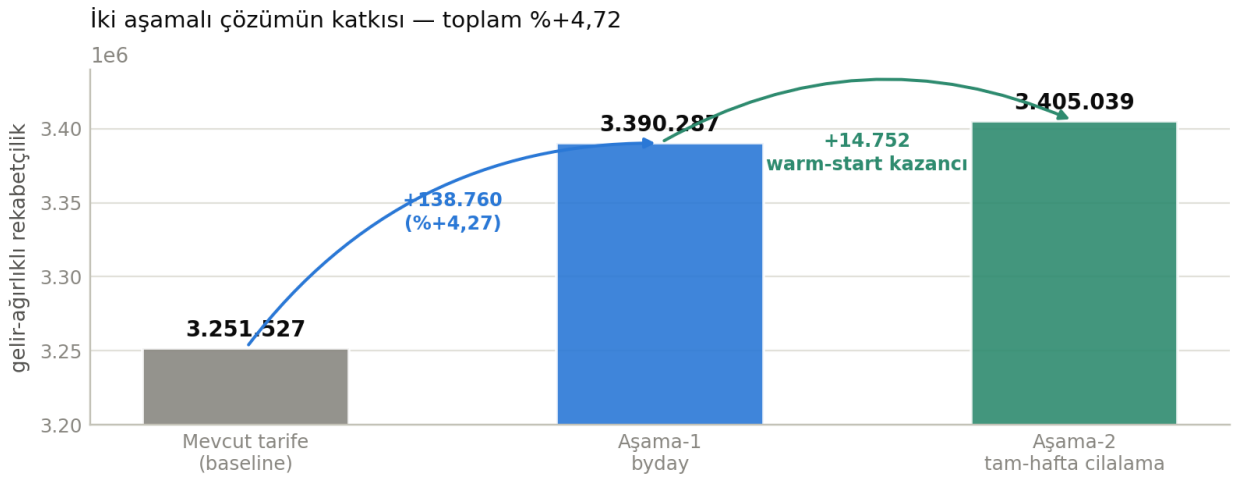
Denedik, olmadı. Tam-hafta modelinde ~100.566 ikili değişken var ve HiGHS makul sürede **tek bir uygun çözüm bile** bulamıyor. Çözüm bulunamayınca budama yok, budama yoksa ilerleme yok.

Çözümümüz iki aşamalı: (1) haftayı günlere böl, her günü ayrı çöz — hızlı ve garantili; (2) bu çözümü tam-hafta modeline başlangıç noktası ver, çözücü küresel olarak iyileştirsin.

S26 Warm-start nedir, ne kazandırdı?

Çözücüye sıfırdan başlamak yerine "*işte elimde çalışan bir çözüm, buradan devam et*" demek. Çözücü onu incumbent kabul edip iyileştirmeye başlıyor.

	Değer
Aşama-1 (byday)	3.390.287
Aşama-2 (tam-hafta cilalama)	**3.405.039**
Warm-start kazancı	**+14.752**
Ayrıca elde edilen	**%1,18 global MIP-gap** — optimalite kanıtı



Şekil 6.3 — İki aşamalı çözümün katkısı.

S27 MIP-gap %1,18 tam olarak ne anlama geliyor?

"*Bulduğum çözüm, teorik olarak mümkün olan en iyinin en fazla %1,18 altında.*" Çözücü hem bir çözüm buluyor hem de bir **üst sınır** kanıtlıyor; gap ikisi arasındaki bağıl fark.

Neden bu, projenin en değerli çıktısı?

Elle yazılmış hiçbir açgözlü algoritma bu cümleyi kuramaz. Greedy bir yöntem size bir tarife verir ama o tarifenin optimale yakın mı çok mu uzak olduğu konusunda hiçbir bilgi vermez. Gap, "*iyi bir çözüm buldum*" ile "*bulduğum çözümün ne kadar iyi olduğunu kanıtlayabiliyorum*" arasındaki farktır.

S28 Daha uzun süre verseniz daha iyi sonuç çıkar mı?

Muhtemelen biraz — gap %1,18 olduğu için teorik tavan yaklaşık %1,2 daha yukarıda. Ama iyileşme giderek yavaşlar (grafikteki yakınsama). Teslim koşusu 1.800 saniye ile sınırlandı; bu, tekrar-üretilebilirlik ve makul koşu süresi arasındaki denge noktası.

S29 Sonuç başka bir makinede aynı çıkar mı?

Evet, tasarım gereği: tek iş parçacığı, sabit rasgele tohum, sabit MIP-gap ve süre sınırları, veriden türetilen (hardcode edilmeyen) gün listesi.

Tek değişken, süre sınırına takılan koşuların makine hızına bağlı olması — bu yüzden gap ve süre sınırları raporlarda **açıkça** yazılıyor.

S30 Çözücü kaç ihtimali gerçekten inceliyor?

Karar uzayı $13^{4553} \approx 10^{5.072}$. Çözücü bunların hepsini değil, dallanma ağacında **birkaç bin ile birkaç yüz bin arası düğümü** inceliyor. Geri kalanı doğrusal gevşeme + budama ile **hiç ziyaret edilmeden** eleniyor.

D · Veri

S31 Veriyi temizlediniz mi? Ne yaptınız?

Temizlemedik — **düzelttik**. Politikamız dört ilkeye dayanıyor: orijinale dokunma, değer uydurma, yalnız fiziksel olarak imkânsız olanı düzelt, her dönüşümü logla. Yaptığımız her işlem `derived/transform_log.md` içinde.

- Excel'in altı farklı süre biçimini tek çeviriciyle dakikaya normalize ettik
- 24 saat sarmasını çözüp J'yi **kesin** hesapladık (mod-24 çapraz doğrulaması: 57.317 satırda 0 tutarsızlık)
- Hafta kaymasını hizaladık (132 satır geri çekildi, 130 kopya olay birleşti)
- Birebir özdeş 157 satırı tekilleştirdik
- Bacak blok sürelerini çıkardık (5.471 kayıt; çok-değerli 173 grupta ihtiyatlı olan alındı)
- Rakip sürelerini taşıyıcı bazında özetledik (49.467 → 23.975)

Yapmadıklarımız: aykırı değer atmadık, eksik değer doldurmadık, normalizasyon/ölçekleme yapmadık, rakip verisine müdahale etmedik, orijinal Excel'lere hiç yazmadık.

S32 24 saat sarması nedir? Nasıl fark ettiniz?

`Gate-to-Gate Uçuş Süresi` sütunu bir **saat hücresi** olarak kaydedilmiş. Saat hücresi 00:00–23:59 arasıdır; 24 saati aşan bir süreyi gösteremez ve başa sarar. 28 saatlik bir yolculuk "04:00" görünür.

Fark etme yöntemi: sütunun aldığı **en büyük değer 1.435 dakika (23:55)** — 57.317 satırda tek istisna yok. Böyle bir tavan doğal olamaz.

Gerçek örnek — EZE→PEK (Buenos Aires → Pekin), TK 16 + 88

Excel diyor: **04:40**. Gerçek: 1. bacak 17 saat + bekleme 2,5 saat + 2. bacak 9 saat = **28,7 saat**. Fark tam 24 saat.

Bu değeri kullansaydık model "Buenos Aires'ten Pekin'e 4 saat 40 dakikada gidiyoruz, herkesi yendik" derdi.

S33 ElapsedTime sütunlarına neden güveniyorsunuz?

Üç ayrı gerekçeyle:

1. **Yapısal:** bu sütunlar `'30.12.1899 06:19:00'` biçiminde **metin** olarak saklanmış — Excel'in 1899 epoch kodlaması. Tarih kısmı 24 saat üstünü taşıyabildiği için biçimin **tavanı yok**.
2. **Fiziksel:** tek bir uçuş bacağı zaten 24 saati aşmaz (en uzunu 23 sa 45 dk, ortanca 2,6 saat). Aşan şey **toplam yolculuk**, ve o zaten farklı bir sütunda.
3. **Çapraz doğrulama:** `(E1 + bekleme + E2)` ile `Gate-to-Gate` arasındaki fark tam olarak 24 saatin katı olmalı. **57.317 satırda 0 tutarsızlık.** İki bağımsız kaynak birbirini tutuyor.

S34 24 saati aşan satırları attınız mı?

Hayır, **tek satır bile atılmadı**. 495 uzun yolculuğun (61 THY + 434 rakip) hepsi doğru J değeriyle modele girdi. Çözümde 61 sunulan bağlantının J'si 24 saati aşıyor ve bunlar toplam 64 rakip yeniyor — bu galibiyetler gerçek.

S35 Eksik veriyi nasıl ele aldınız?

İki tür eksik vardı:

Eksik	Kaç	Politikamız	Neden
ρ (gelir ağırlığı) olmayan pazar	48	Amaç evreninden **dışla** — ama uçuşları kapasitede saymaya devam et	0 saymak "değersiz ilan etmek", ortalama atamak **veri uydurmak** olurdu
Rotasyon partneri olmayan uçuş	176	**Sabit** kabul et, hiç kaydırma	Doğrulayamadığımız bir kısıtı bozma riski almıyoruz

S36 Gizli test verisi farklı gelirse kod çalışır mı?

Bunun için özel olarak hazırlandı:

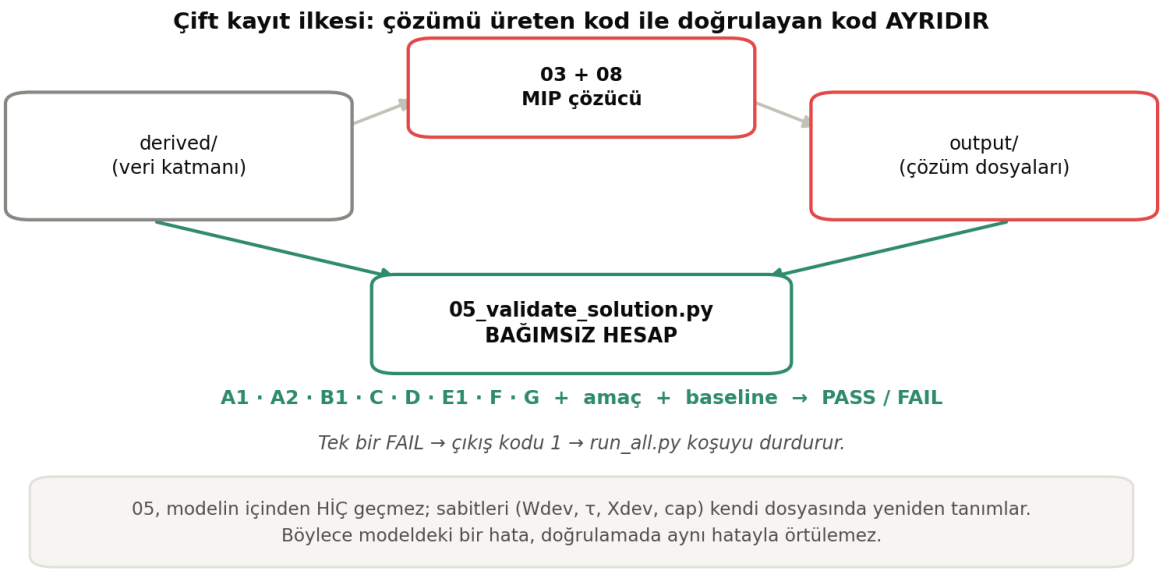
- Gün listesi veriden okunuyor, hardcode edilmiyor (`all_operation_days()`)
- Dosya adı farklıysa `glob` yedeğiyle aranıyor (`_find_conn_file()`)
- Beklenen sütun yoksa **anlaşılır** hata veriyor (`need()`), kriptik `KeyError` değil
- `ElapsedTime` sütunları yoksa eski şemaya (minimal de-wrap) otomatik düşüyor ve bunu loglara yazıyor
- `flight_block_times.csv` yoksa A2 ihtiyatlı moda geçiyor
- Tek bir günün çözümü hata verirse o gün atlanıyor, koşu devam ediyor

E · Doğrulama ve güvenilirlik

S37 Sonucunuzun doğru olduğunu nasıl biliyoruz?

Üç bağımsız kanıtla:

1. **Bağımsız doğrulayıcı.** `05_validate_solution.py` modelin kodundan tamamen ayrı — Pyomo import etmiyor, sabitleri kendi dosyasında yeniden yazıyor, baseline'ı sıfırdan hesaplıyor. 12 kontrolün hepsi PASS.
2. **Global MIP-gap %1,18.** Çözücü, çözümün optimale ne kadar yakın olduğunu matematiksel olarak kanıtlıyor.
3. **Kalite kapısı.** Doğrulayıcı bir tane bile FAIL bulursa çıkış kodu 1 döndürüyor ve `run_all.py` koşuyu durduruyor. Yani `output/` klasöründe bir sonuç varsa, o sonuç kesinlikle 12 kontrolden geçmiştir.



Şekil 6.4 — Çift kayıt: üreten kod ile doğrulayan kod ayrıdır.

S38 Doğrulayıcı gerçekten bir hata yakaladı mı?

Evet — ve bu, sunumdaki en önemli hikâyemiz.

1. Pencereyi ±90 dakikaya çıkardık, sonuç **%5,34**'e yükseldi. Cazip bir rakam.
2. Doğrulayıcı **gerçek bir A2 rotasyon ihlali** yakaladı (uçuş 2200/2201, gün 2).
3. Kök nedeni bulduk: rotasyon ortağının o günkü olayı ρ'suz bir pazara ait olduğu için modelde hiç oluşturulmuyordu → A2 kısıtı o gün hiç kurulmuyordu.
4. **Yüksek rakamı iptal ettik**, ±60'a döndük ve kök nedeni (`full_role_time` evren-ankoru ile) kalıcı olarak düzelttik.

Bu not `05_validate_solution.py`'nin 52–60. satırlarında hâlâ duruyor — silmedik, çünkü kararın gerekçesi kodun bir parçası.

S39 G muafiyetleri bir açık kapı değil mi?

Olabilirdi — ama muafiyet listesi **diske yazılıyor** (`output/g_band_exemptions.csv`, 1.132 kayıt), raporlarda sayısıyla anılıyor ve doğrulayıcı onu okuyup ayrıca sayıyor.

Alternatif — muafiyetleri hiç bildirmemek — G'nin nerede uygulanmadığını **bilinemez** kılardı. Şeffaf gevşetme, gizli gevşetmeden iyidir.

S40 "Artefakt" dediğiniz A2 ihlalleri gerçek ihlal olabilir mi?

Artefakt olarak sayılan turlar, **mevcut yayınlanan tarifenin kendisinin de sağlamadığı** turlardır — yani o gün o çift zaten tek uçakla dönülmüyor. Optimizasyon onları yaratmadı, devraldı.

Kod ikisini asla karıştırmıyor: `a2x_base` (artefakt) ve `a2x_viol` (YENİ ihlal) ayrı sayılıyor ve **ikisi de** raporlanıyor. YENİ ihlal sayısı 0 olmak zorunda — ve 0. Ayrıca bu, 08'de bir yeniden yazma bariyeri: yeni ihlal doğarsa çıktı dosyalarına dokunulmuyor.

S41 Baseline'ı nasıl hesapladınız? Kendi lehinize olabilir mi?

Baseline **iki kez, iki bağımsız kodla** hesaplanıyor: `03_model.py`'de `current_baseline()` ve `05_validate_solution.py`'de sıfırdan. İkisi uyuşmuyorsa doğrulama FAIL veriyor.

Ayrıca `b_od` tanımını özellikle baseline'ı **şişirmeyecek** şekilde kurduk: mevcut tarife ödül tablosunun köşegenine (nötr referans) oturuyor. Eski kural kullanılsaydı baseline pazarların ~%29'unda hak etmediği ödülü alır ve iyileşmemiz **olduğundan büyük** görünürdü. Yani hatayı kendi aleyhimize düzelttik.

S42 Fazla-beyan ve eksik-beyan neden farklı muamele görüyor?

Fazla-beyan (dosyada yazandan az rakip gerçekten yenilmiş) amaç değerini şişirir → gerçek hata, FAIL. **Eksik-beyan** (gerçekte yenilmiş ama gösterge bağlanmamış) amaç değerini olduğundan küçük gösterir → hata değil, ihtiyatlılık, sadece not edilir.

Bu asimetri bilinçli: **kendi lehimize olan sapmalar hata sayılıyor, aleyhimize olanlar sayılmıyor.** Ölçüm kendine karşı kurulmuş.

F · Sonuç, özgünlük ve eleştiri

S43 %4,72 anlamlı bir kazanç mı?

Evet, üç nedenle:

- **Gelir-ağırlıklı.** Bu bir uçuş sayısı yüzdesi değil; her pazarın gelir potansiyeliyle çarpılmış bir rekabetçilik ölçüsü. Büyük pazarlardaki iyileşme daha ağır basıyor.
- **Çok düşük maliyetle elde edildi.** Uçuşların yalnız %23'ü, ortalama 19 dakika kaydırıldı. %77'si hiç kımıldamadı. Yeni uçak, yeni sefer, ek yatırım gerekmiyor.
- **Kanıtlı.** Global gap %1,18 — yani mümkün olan en iyi bile bizden en fazla %1,2 daha iyi. Masada bırakılan çok az şey var.

Somut sonuçlar: 660 pazar-günde sıralama yükseldi, 4.914 rakip yenildi, 6.966 bağlantı sunuldu.

S44 Çözücüyü kullandınız — sizin katkınız ne?

Çözücü, sorduğunuz sorunun cevabını verir. **Bütün maharet doğru soruyu sormaktır.** Yanlış formüle etseydik çözücü bize yanlış sorunun kusursuz optimal cevabını verirdi — ve bunu fark etmenin hiçbir yolu olmazdı.



Şekil 6.5 — Beş adımın dördü bizim tasarımımız.

Katkımız beş başlıkta: **formülasyon kararları** (olay soyutlaması, one-hot kodlama, sıkı big-M, üç yollu değişken elemesi, b_od türetimi), **çözüm stratejisi** (iki aşamalı matheuristik, gece-aşan koruma, G zincirlemesi), **veri mühendisliği** (kesin J, hafta normalizasyonu, evren-eksiği düzeltmeleri), **bağımsız doğrulama** ve **dürüst raporlama**.

En güçlü kanıt Bölüm 3'teki "yapmasaydık ne olurdu" tablosu: her satır ölçülmüş bir tasarım kararı ve hiçbiri çözücüden gelmiyor.

S45 Bu teknikler standart değil mi? Neyi siz buldunuz?

Dürüst cevap: **teknikler standart, uygulaması bize ait.** Big-M, one-hot kodlama, azalan getiri, warm-start — hepsi bilinen yöneylem araştırması araçları. HiGHS'in algoritması ve Pyomo da bizim değil.

Bizim kattığımız, bu araçların **bu probleme nasıl uygulanacağına** dair tasarım kararları: karar biriminin ne olacağı, hangi değişkenin hiç yaratılmayacağı, M'lerin nasıl sıkılaştırılacağı, çözülemeyen modelin nasıl ikiye bölüneceği, gün ayrıştırmasının gece-aşan turları nasıl bozmayacağı ve sonucun nasıl bağımsız doğrulanacağı.

Kapanış cümlesi

"Biz bir yapay zekâ modeli eğitmedik — bu bir tahmin değil, karar problemi. Yaptığımız şey brief'teki iş problemini 100.566 ikili değişkenli bir matematiksel modele çevirmekti; karar biriminden ödül yapısına, big-M sıkılığından kısıt formülasyonuna kadar her tasarım kararı bize ait. Çözücü bu modeli çözen motor — ama sorduğumuz soru yanlış olsaydı bize yanlış sorunun kusursuz cevabını verirdi. Doğru sorduğumuzun kanıtı üç yerde: modelden tamamen bağımsız yazdığımız doğrulayıcı 12 kontrolün hepsini geçiyor, çözücü %1,18'lik global optimalite boşluğunu kanıtlıyor, ve daha yüksek sonuç veren bir denememizi kendi doğrulayıcımız ihlal yakaladığı için iptal ettik."